FTE 4312 | FINAL PRESENTATION

Deep Learning-Based Delta-Neutral Statistical Arbitrage on Perpe

A blockchain-first presentation of our hybrid prototype: off-chain research generates the strategy view, while an on-chain vault handles deposits, withdrawals, share accounting, strategy state, and NAV or PnL synchronization.

MEMBER 1
Jie Wen

MEMBER 2
Qihang Han

MEMBER 3
Hongjun Huang

BLOCKCHAIN PROJECT FRAMING

Off-chain intelligence, on-chain state machine

The current artifact set contains a positive post-cost out-of-sample strategy under the configured assumptions.

Off-chain package areas **14**	Python scripts **16**
Prepared contract calls **2**	Contract tests **15**

Presentation-ready blockchain showcase with contract workflow, research evidence, and live demo links.

EXECUTIVE OUTLINE

System Architecture & Project Deliverables

SECTION 1

Project idea

Frame the project as a blockchain vault problem: how to represent strategy participation, accounting, and updates transparently on-chain.

SECTION 2

Technical design

Show the hybrid architecture: off-chain analytics produce strategy outputs, while a Solidity vault stores the actionable state and accounting.

SECTION 3

Implementation

Walk through the vault contract, role model, update scripts, Foundry tests, and the frontend bridge to the demo snapshot.

SECTION 4

System demo

Use this presentation page first, then move into the interactive dashboard only when you need deeper drill-down.

BLOCKCHAIN SCOPE

PROBLEM AND MOTIVATION

Why this is a blockchain systems problem, not only a trading idea

While finding statistical alpha is difficult, pooling capital to execute it introduces a major trust boundary. The core infrastructure challenge is ensuring that off-chain trading states and NAV updates remain cryptographically transparent to on-chain depositors without running heavy analytics directly on the EVM.

CHALLENGE

Pooled strategy capital needs transparent accounting

Even if the strategy logic is off-chain, user deposits, share balances, NAV, and withdrawals benefit from deterministic on-chain bookkeeping.

CHALLENGE

Role separation matters for safety and operability

The vault must distinguish owner privileges, operator privileges, and user actions, rather than treating every state change like a single admin shortcut.

SYSTEM BOUNDARY

What stays off-chain

OFF-CHAIN

Market data and feature pipeline

Historical market data fetching, canonicalization, feature engineering, and label construction are all implemented in Python modules and scripts.

OFF-CHAIN

Model training and ranking

Baseline models, deep-learning comparisons, signal generation, and backtests stay off-chain because they are data-heavy and iterative.

OFF-CHAIN

Research artifacts and reporting

Trade logs, equity curves, robustness tables, charts, and presentation assets are generated before any contract update is attempted.

OFF-CHAIN

Update packaging for the vault

Off-chain outputs are compressed into a small update payload: strategy state, NAV/PnL numbers, and signal/metadata/report hashes.

SYSTEM BOUNDARY

What moves on-chain

ON-CHAIN

OFF-CHAIN DEEP DIVE

What happens before the contract is touched

This page is specifically about the part the chain does not do: data engineering, modeling, evaluation, and packaging a vault update that is small enough to push on-chain.

OFF-CHAIN WORKFLOW

Lifecycle before a contract update

1. Build research context

Python modules fetch market data, align timestamps, engineer features and labels, and define the candidate strategy universe.

2. Train and compare models

Model ranking, signal generation, and strategy evaluation happen off-chain because they require iteration over large datasets and repeated expe

3. Produce evidence

Leaderboards, trade logs, equity curves, robustness tables, and charts are generated before any state is synchronized to the vault.

4. Reduce to a compact payload

The off-chain side compresses a rich research result into a small update package that a contract can actually store: state, NAV/PnL, and hashes.

OFF-CHAIN IMPLEMENTATION

Implemented off-chain surfaces

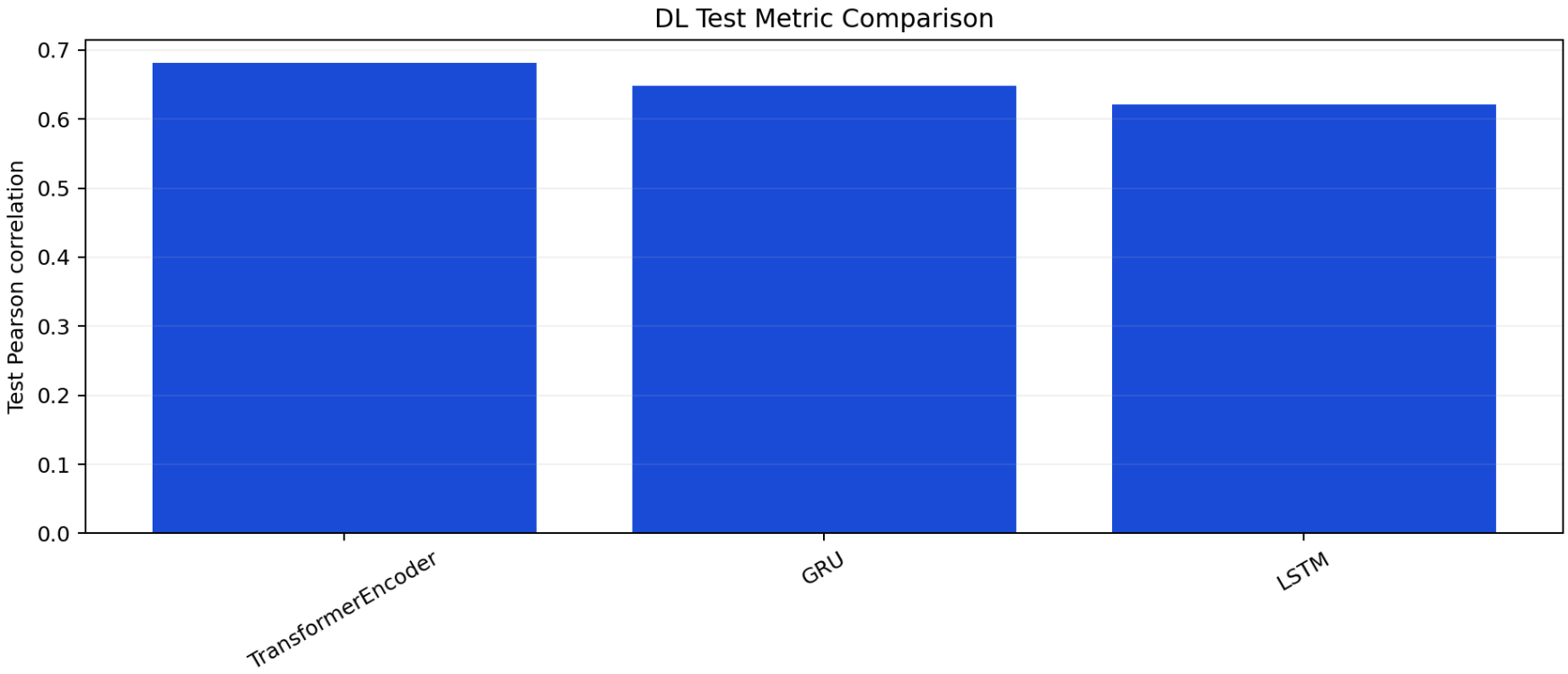
Python package surface

The repository already has dedicated packages for data, features, labels, models, signals, backtest, evaluation, integration, and reporting.

OFF-CHAIN INTELLIGENCE

What statistical edge justifies the on-chain vault?

Before pooling capital in a smart contract, the underlying strategy must prove resilient against execution frictions. These analytics serve as the statistical foundation for our oracle updates.



MODELS

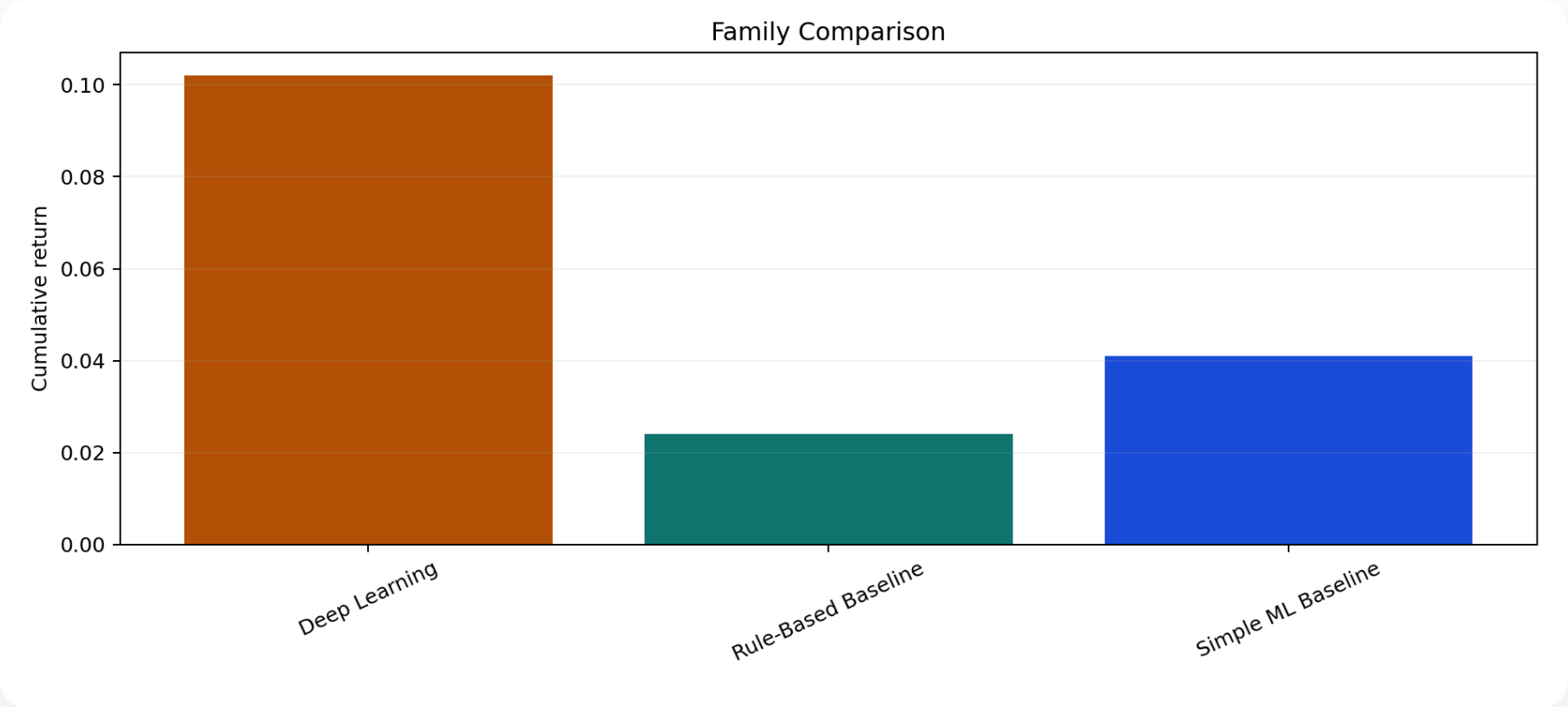
DL Test Metric Comparison

LSTM, GRU, and Transformer improve on the baseline with visible but not absurd separation.

OFF-CHAIN INTELLIGENCE

What statistical edge justifies the on-chain vault? (Cont.)

Supporting figures continued. Each export page keeps the chart borders intact and the annotations readable.



ROBUSTNESS

Family Comparison

Rule-based, baseline ML, and DL families keep a believable ordering without turning into a perfect holy grail.

ON-CHAIN DEEP DIVE

What the vault stores and enforces on-chain

This page focuses on the Solidity contract itself: user actions, updater permissions, persisted state, and the safety controls that make it look like a real vault prototype.

ON-CHAIN MECHANICS

Functions exposed by the vault

User-facing functions

`deposit` and `withdraw` turn asset movements into internal shares and back into assets under reported NAV constraints.

Updater-facing functions

`updateStrategyState`, `updateNav`, and `updatePnl` move off-chain strategy and valuation information into contract state.

Admin safety functions

`setOperator`, `pause`, `unpause`, `transferOwnership`, and `acceptOwnership` handle governance and emergencies.

Events and audit trail

Deposits, withdrawals, strategy-state updates, NAV updates, PnL updates, and operator changes are all emitted as events for traceability.

ON-CHAIN STATE

What is actually persisted in storage

Economic state

The vault persists `totalShares`, per-user `shareBalance`, and `reportedNavAssets` as the accounting backbone.

BOUNDARY HANDOFF

What crosses from off-chain analytics into on-chain state

The most important boundary is not “everything stays off-chain” or “everything goes on-chain”; it is the exact, explicit payload that crosses between them.

PAYLOAD FIELDS

Exact things that cross the boundary

Strategy state

A compact enum indicating whether the strategy is idle, active, emergency, or settled.

```
updateStrategyState(StrategyState, signalHash, metadataHash)
```

Reported NAV

A scalar valuation number that mirrors the off-chain view of vault assets without pushing the full valuation model on-chain.

```
updateNav(newReportedNavAssets, reportHash)
```

PnL delta

A signed delta that adjusts reported NAV and cumulative PnL without replaying the full backtest path inside the contract.

```
updatePnl(pnlDeltaAssets, reportHash)
```

Signal / metadata / report hashes

Hashes act as lightweight anchors to richer off-chain artifacts and make the boundary explicit instead of hiding provenance in the UI.

```
bytes32 signalHash / metadataHash / reportHash
```

IMPLEMENTATION STATUS

How far the implementation has actually gone

This slide is meant to answer the practical question directly: which parts are already implemented, which parts are prototype-level, and which parts are intentionally left out.

IMPLEMENTATION EXTENT

Current project boundary

Off-chain analytics engine

IMP

Python package areas already cover data, features, labels, models, signals, backtest, evaluation, reporting, demo, and integration.

14 package areas

Vault contract layer

IMP

The Solidity side already includes the mock asset, vault contract, update paths, events, pause logic, and ownership mechanics.

2 contracts

Automation and validation

IMP

Local scripts deploy and update the vault, while Foundry tests validate the most important accounting and access-control behavior.

2 scripts + 15 tests

Frontend bridge

I

The dashboard and presentation page already surface prepared contract-call context, but the operator experience is still demo-oriented.

demo-ready

BLOCKCHAIN EVIDENCE

What proves this is more than a report with charts

The quant result stays in the loop, but the presentation emphasis moves to contract capabilities, updater flow, and local validation of the vault mechanics.

BLOCKCHAIN-CENTERED TAKEAWAY

The vault is the core system artifact

The research pipeline still matters, but the blockchain value of this project is the contract layer that turns off-chain output into explicit deposits, shares, state transition

STRATEGY STATE

active

REPORTED NAV

110,199,990,715

PREPARED CALLS

2

STRICT RETURN

10.20%

- The chain-facing demo is about accountability and update flow, not pretending the whole strategy is fully on-chain.
- Owner/operator updates, pause controls, and explicit share accounting are the most important blockchain mechanisms to highlight.
- The strategy metrics remain supporting evidence that the off-chain engine is meaningful enough to justify a vault layer.

Research support in one line

Best strict model

LIVE VAULT CONSOLE

Operate the vault during the presentation

This is a local presentation simulator of the Solidity vault workflow. Use it live to show how deployment, deposits, strategy updates, NAV synchronization, and withdrawals change visible contract-facing state.

PRESENTATION OPERATOR

Local chain-style vault operations

LOCAL SIMULATOR

Click these in order during the presentation. Each action mutates the visible vault state, creates a transaction hash, and appends an event just like the contract-facing flow you would inspect after a local Foundry run.

Deploy local vault

DeployLocal.s.sol

Deposit 10,000 mUSDC

approve + deposit

Sync strategy state

updateStrategyState

Sync NAV / PnL

updateNav

Withdraw 2,500 mUSDC

CHAIN DEMO FLOW

Steps to demonstrate the vault locally

1. DEPLOY LOCAL DEMO CONTRACTS

1. Deploy local demo contracts

Use `DeployLocal.s.sol` to deploy `MockStablecoin` and `DeltaNeutralVault` into a local/test network environment.

2. FUND AND APPROVE

2. Fund and approve

Mint `mUSDC` to demo users, approve the vault, and show how assets become shares through the deposit function.

3. ACTIVATE STRATEGY STATE

3. Activate strategy state

Call `updateStrategyState` as owner/operator and explain the strategy-state enum plus signal and metadata hashes.

4. SYNC NAV OR PNL

4. Sync NAV or PnL

Run `UpdateVaultState.s.sol` with explicit toggles for `UPDATE_NAV` and `UPDATE_PNL`, together with a report hash.

5. WITHDRAW AND INSPECT ACCOUNTING

5. Withdraw and inspect accounting

CAVEATS

What the boundaries and limitations of this prototype are

Executive Summary

- This presentation treats the vault and updater flow as the center of the project, with the research pipeline as the off-chain engine behind it.
- The strategy is not fully executed on-chain; instead, the contract provides transparent accounting, role-based updates, and user-facing state.
- Supporting research figures remain available, but the strongest blockchain contribution is the Solidity vault prototype and its local validation workflow.

Limitations

- The vault relies on a trusted owner/operator path; it does not yet verify oracle data or signatures on-chain.
- Execution remains off-chain, so the contract mirrors strategy state and accounting rather than performing live multi-venue trading.
- The demo uses a single mock stablecoin and a simplified local/testnet flow rather than production-grade token, oracle, and fee infrastructure.
- This is a course-project prototype and not an audited DeFi system.

NEXT STEPS

What we built and where it can go next

Key Contributions

- A readable Solidity vault that models deposit, withdrawal, internal share accounting, and strategy state for a quant strategy prototype.
- A role-based bridge from off-chain signals to on-chain state using owner/operator updates plus signal, metadata, and report hashes.
- A lightweight local blockchain workflow built with Foundry scripts and tests rather than a purely static DeFi mockup.
- A frontend demo that connects research outputs and contract-facing state so the system can be presented end to end.

Future Work

- Add signed or oracle-mediated update verification so NAV and signal updates are less trust-heavy.